# CardioIA Fase 7 — Roteiro do Vídeo Demonstrativo

Nov 10, 2026 · @Carlos

## Objetivo

O vídeo tem **4 min 30 s** e mostra um dado saindo do sensor no Wokwi e chegando à URL pública da Vercel e ao APK, sem cortes no momento da atualização. Os 30 s que sobram do limite de 5 min são folga para carregamentos.

O enunciado pede: demonstração do fluxo fim a fim, do hardware (Wokwi) até a atualização do dado na Vercel/APK, em até 5 minutos. O trecho que mais pesa na nota é a cena 6 (risco alto), em que a banca vê o dado mudar em tempo real.

## Preparação antes de gravar

- [ ] Abrir o /health do backend **5 minutos antes** e confirmar `"status":"ok"` (o Render gratuito hiberna).
- [ ] Deixar quatro abas abertas, nesta ordem: Wokwi, Vercel já logada (`admin@cardioia.com` / `CardioIA2026`), /health e o diagrama de arquitetura.
- [ ] Abrir o emulador Android com o APK instalado e logado.
- [ ] No Wokwi, deixar os potenciômetros na posição inicial (BPM ~75, $SpO_2$ ~94 %) e a simulação **parada**.
- [ ] Configurar a gravação em 1080p, com zoom do navegador entre 110 % e 125 %.
- [ ] Deixar Wokwi e Vercel lado a lado (tela dividida) para a cena 6.

## Roteiro cena a cena

São 9 cenas. A narração é uma sugestão: adapte ao seu jeito de falar, mantendo os números.

| # | Tempo | O que aparece na tela | Narração sugerida |
|---|---|---|---|
| 1 | 0:00–0:20 | Capa do README ou slide com o nome do projeto e da equipe | "Este é o CardioIA, Fase 7. Vamos mostrar o fluxo completo: um sensor simulado no Wokwi envia sinais vitais para o backend Python, a IA calcula o risco, e o resultado aparece na Web e no celular." |
| 2 | 0:20–0:50 | Diagrama de arquitetura, passando o mouse por cada bloco | "O ESP32 roda MicroPython e envia leituras a cada 5 segundos ao backend FastAPI no Render. O backend usa o modelo preditivo da Fase 6, aplica os protocolos clínicos e devolve o risco. A Web na Vercel e o app Android consultam a API a cada 5 segundos." |
| 3 | 0:50–1:10 | Aba do /health e, rapidamente, o /docs (Swagger) | "O backend está no ar: o modelo está carregado e o assistente configurado. Aqui estão as rotas da API." |
| 4 | 1:10–1:50 | Wokwi: circuito e topo do main.py (BACKEND_URL). Clicar em Start e mostrar o console | "No circuito temos o DHT22 para temperatura, um potenciômetro para batimentos e outro para saturação, dois LEDs e um display OLED. Ao iniciar, o ESP32 conecta ao Wi-Fi e envia a primeira leitura. O LED verde indica que o envio deu certo, e o OLED mostra o risco devolvido pelo backend." |
| 5 | 1:50–2:20 | Vercel com ~75 BPM e risco baixo | "Na URL pública da Vercel, o painel já mostra a leitura do esp32-wokwi-01: batimentos normais e risco baixo, em verde." |

| # | Tempo | O que aparece na tela | Narração sugerida |
|---|---|---|---|
| 6 | 2:20–3:10 | **Momento principal.** No Wokwi: BPM ~145, $SpO_2$ ~77 %, DHT22 ~39 °C. Mostrar console (ALERTA e Risco 0.86 alto), OLED e os dois LEDs acesos. Depois, a Vercel atualizando | "Agora simulamos uma crise: taquicardia, saturação baixa e febre. O ESP32 já gera o alerta localmente, e o backend devolve risco de 86 %, faixa alta. O LED vermelho acende. Na Vercel, sem recarregar a página, o painel muda para risco alto e lista os protocolos: suporte respiratório, monitoramento intensivo, protocolo infeccioso e estado de emergência." |
| 7 | 3:10–3:45 | Emulador com o app mostrando a mesma leitura; depois, a página do build no Expo (Succeeded) | "O mesmo dado chega ao aplicativo Android, gerado pelo EAS Build e instalado a partir deste APK. As duas interfaces consomem a mesma API." |
| 8 | 3:45–4:10 | Wokwi com os sinais de volta ao normal. Opcional: OLED com OFFLINE Fila 3 e a sincronização [SYNC] no console | "Ao normalizar os sinais, o risco volta a baixo. Se o Wi-Fi cair, o ESP32 guarda até 20 leituras e as envia em lote quando a conexão volta, sem perder dados." |
| 9 | 4:10–4:30 | README no GitHub com a tabela de links | "Os links da Vercel, do APK, do Wokwi e o código estão no README do repositório. Este é um protótipo acadêmico com dados simulados. Obrigado." |

## Dicas de gravação e publicação

- **Use os valores da cena 6.** BPM ~145 (potenciômetro em ~769), $SpO_2$ ~77 % (em ~240) e 39 °C foram conferidos com o modelo e dão risco de 86 %. Evite improvisar: com 169 BPM e $SpO_2$ 94 %, o risco fica baixo (18 %), o que confunde quem assiste.
- **Espere a atualização.** O painel atualiza a cada 5 s. Fale durante a espera em vez de cortar, para ficar claro que é tempo real.
- **Mantenha a aba do Wokwi visível.** Em segundo plano, a simulação fica muito lenta.
- **Grave por partes.** Grave cada cena separada e junte no final com um editor simples, como o Clipchamp do Windows. Assim um erro não obriga a refazer tudo.

- **Publique como não listado.** Suba no YouTube como "Não listado" e confira se o link abre numa janela anônima.
- **Atualize os documentos.** Coloque o link do vídeo no README (seção 1) e no relatório PDF (tabela de links da página 1).

## Checklist final e responsáveis

Os responsáveis seguem a divisão de funções do grupo; ajuste se combinarem diferente.

| Item | Responsável |
|---|---|
| Confirmar que o código do Wokwi está igual ao do repositório (main.py e logica.py) | João |
| Deixar o APK instalado e logado no emulador | Tayná Esteves |
| Conferir o painel Web e o app antes de gravar | Endrew Alves |
| Gravar, editar e publicar o vídeo (até 5 min) | Carlos Souza |
| Incluir o link do vídeo no README e no relatório PDF | Carlos Souza |